Forgettable Athlete — Detailed Explanation

Why Logistic Regression won, what the SHAP graphs mean, how SISA works, and how to read the correlation heatmap.

1. The Model Ranking (by ROC-AUC)

Rank	Model	Accuracy	F1	ROC-AUC	Fit time (s)
1st	Logistic Regression	0.9500	0.9500	0.9961	0.021
2nd	Random Forest	0.9563	0.9560	0.9925	0.649
3rd	LightGBM	0.9187	0.9161	0.9881	0.152
4th	XGBoost	0.9375	0.9359	0.9848	0.131

Random Forest has slightly higher raw accuracy, but **ROC-AUC is the primary metric** because in injury prediction both false positives (unnecessary rest) and false negatives (missed injury) matter. ROC-AUC measures probability-ranking quality across every threshold, not just hard predictions at 0.5.

2. Why Logistic Regression Wins — the Theory

Logistic Regression draws a **straight line (hyperplane) through the feature space** and says: everyone on this side is at injury risk, everyone on the other side is safe. It only works well when the signal in the data is roughly linear.

A. Your engineered features pre-cooked the non-linearity.

You manually built four features:

- **ACWR** = training load relative to a rolling 5-row mean (a ratio).
- **Recovery_Score** = weighted blend of sleep + nutrition + warmup minus stress.
- **Fatigue_Index** = blend of training load + matches minus recovery.
- **Injury_Frequency_Score** = previous injuries divided by age.

These are already non-linear combinations of the raw columns (ratios, weighted sums). By the time the linear model sees them, the hard work is done — it just needs to draw a line through them. Tree models (RF, XGBoost, LightGBM) try to **rediscover** these patterns from scratch by splitting raw columns repeatedly, which on only 640 training rows wastes capacity and risks overfitting.

B. Small dataset (n = 640 train).

Boosting algorithms shine with tens of thousands of rows. On small tabular data with strong linear signals, the simpler model usually wins — this is a textbook **bias-variance tradeoff** outcome. Trees have high variance; linear models have low variance. Less data means variance hurts more.

C. Calibrated, smooth probabilities.

Logistic outputs smooth probabilities via the sigmoid function. ROC-AUC rewards good probability *ranking*, which is exactly what logistic regression is mathematically optimised for.

Why the others ranked where they did

- **Random Forest (2nd)** — Averages many decision trees, so variance drops vs a single tree. It found most of the signal but slightly overfits because trees can't represent a smooth ratio like ACWR cleanly — they approximate it with many step-function splits.
- **LightGBM (3rd)** — Uses *leaf-wise* growth (always splits the leaf with highest loss reduction). Very aggressive — great on big data, but on 640 rows it overfits noisy splits.

• **XGBoost (4th, worst)** — Similar to LightGBM but with level-wise growth. Without hyper-parameter tuning, its defaults didn't help on a dataset this small.

In short: the dataset is small + linear-friendly + already feature-engineered → linear model wins.

3. Which Features Did the Heavy Lifting

From the top-10 correlations and SHAP, the dominant features are:

1. **Fatigue_Index** — workload pressure offset by recovery. Higher → more injury.
2. **Recovery_Score** — sleep + nutrition + warmup. Higher → less injury.
3. **Previous_Injury_Count** — past injuries predict future injuries.
4. **Injury_Frequency_Score** — chronic injury-prone players, age-adjusted.
5. **Training_Hours_Per_Week** and **ACWR** — workload spikes are dangerous.
6. **Sleep_Hours_Per_Night** and **Stress_Level_Score** — recovery quality.

The **engineered features did most of the predicting** — that is why logistic regression worked so well. Raw features like BMI, height, or position alone carried little signal.

4. The Correlation Heatmap — Simple Explanation

A correlation heatmap shows how each pair of variables moves together, with values from **-1 to +1**:

- **+1 (deep red):** when one goes up, the other goes up. Perfect positive link.
- **0 (white):** no relationship.
- **-1 (deep blue):** when one goes up, the other goes down. Perfect negative link.

How to read your heatmap

1. **Look at the Injury_Next_Season row/column** — that's your target. The cells there tell you which feature is the strongest predictor. The notebook says *no single feature dominates*, meaning all correlations with the target are modest (under 0.3 each). This is normal for injury data — injuries are multi-causal.

2. **Big red blocks among physical tests** — features like Knee_Strength_Score, Balance_Test_Score, Agility_Score, Sprint_Speed_10m_s are correlated with each other. That means they carry overlapping information (a strong athlete is strong on all of them). Models will use one and ignore the others — this is called **multicollinearity**.

3. **Fatigue_Index and Recovery_Score show stronger correlation with the target** than the raw features — proof that feature engineering added real signal.

4. **BMI is essentially flat with the target** — your EDA confirmed this; the BMI distributions overlap fully across injured vs non-injured players.

One-line takeaway: No single feature is a smoking gun — injury comes from a *combination* of fatigue, prior injuries, and poor recovery, which is why the engineered composite features beat the raw ones.

5. SHAP Analysis — What the Graphs Mean

SHAP (SHapley Additive exPlanations) comes from game theory. Imagine each feature is a player on a team, and the team's job is to predict whether a footballer will get injured. SHAP fairly distributes the 'credit' for the prediction across all features — for every single prediction.

For each prediction, SHAP says: 'the model would normally predict X (the baseline), but for this specific player, feature A pushed the prediction up by +0.15, feature B pushed it down by -0.08…' and so on.

The Summary Plot (beeswarm — first SHAP graph)

This is the dot-cloud chart. Read it like this:

- **Y-axis** = features, sorted from most important (top) to least.
- **X-axis** = SHAP value. Right of zero pushes prediction toward 'Injury'; left pushes toward 'No injury'.
- **Each dot** = one player in the test set.
- **Dot colour** = the actual value of that feature for that player. **Red = high value, Blue = low value.**

How to interpret a row:

If red dots cluster on the **right** and blue on the **left**, then high values of that feature increase injury risk (positive relationship). For Fatigue_Index, red dots will sit on the right → high fatigue → more injury risk. For Recovery_Score, you'll see the opposite: red on the left → good recovery protects players.

The Bar Plot (second SHAP graph)

Just the **average absolute SHAP value** per feature — a clean ranking of 'how much does this feature move predictions, on average?' The top bars are your most influential features overall.

The Waterfall Plot (third SHAP graph — for one player)

Explains ONE specific injured player's prediction:

- Starts at the model's **baseline probability** (about 50% in your balanced dataset).
- Each **red bar** = a feature pushing the prediction up toward 'Injury'.
- Each **blue bar** = a feature pushing the prediction down toward 'No injury'.
- Ends at the model's **final prediction** for that player.

Why this matters: A team doctor can look at this plot and say 'We're resting this player because his Fatigue_Index is +0.18, his Previous_Injury_Count contributes +0.12, and his low Recovery_Score adds another +0.09 — here's the receipt.' That's auditable medicine, not a black box.

6. SISA — Machine Unlearning Explained Simply

SISA = Sharded, Isolated, Sliced, Aggregated. It's a clever trick for handling 'right to be forgotten' requests (GDPR) without retraining your whole model.

The Problem It Solves

A footballer leaves the club and says 'delete my data.' Normally you would:

1. Remove their rows from the training data.
2. **Retrain the entire model from scratch.**

On big datasets this is expensive — hours, possibly days. SISA shortens it.

How SISA Works (4 Steps)

1. **Shard** — Split your training data into N independent chunks (you used N=5 shards of 128 players each).
2. **Isolate** — Train **one separate model per shard**. Each model only ever sees its own 128 players, nothing else.
3. **Slice** (optional) — Inside each shard, save intermediate checkpoints so you can roll back to a midpoint instead of starting over.
4. **Aggregate** — At prediction time, all 5 shard-models vote (average their probabilities). That's your ensemble prediction.

Why It's Fast for Deletions

When a player asks to be forgotten:

- Find which shard contains them.
- **Retrain only that one shard.** The other shards never saw them, so they don't need to change.
- Cost = 1/N of a full retrain (in the best case).

Your Notebook's Result

Method	Accuracy	ROC-AUC	Time (s)
Full retraining	0.9563	0.9922	0.148
SISA unlearning	0.9375	0.9855	0.116

- 54 players requested deletion in the training set.
- **Time saving: 21.9%** — and post-unlearning ROC-AUC is virtually identical to full retraining.

The 22% saving looks small here because the dataset is tiny (640 rows, total time 0.15 seconds). In a real club with **100,000+ rows and gradient-boosted models**, full retraining takes minutes-to-hours, and SISA gives massive time savings (often 80%+). In your specific run, all 5 shards happened to contain at least one deletion request, so you retrained all 5 anyway — with luckier sharding or more shards, the saving grows.

The Tradeoff

- **Pro:** Cheap deletions, provable forgetting.
- **Con:** Each shard sees less data → individual shards are weaker (your shards each got about 89–91% accuracy vs the 95% full model). The ensemble averaging mostly recovers performance, but there's a small permanent accuracy cost.

7. The Big Picture

Your pipeline elegantly combines three things that are usually separate:

1. **Sports science** — workload, recovery, fatigue features grounded in real injury research.
2. **Explainable AI** — SHAP makes the model auditable for medical staff.
3. **Privacy compliance** — SISA gives you GDPR-style 'right to be forgotten' without burning compute.

The reason Logistic Regression won is not that it's a 'better algorithm' in general — it's that for this particular setup (small dataset + well-engineered linear-friendly features + the need for calibrated probabilities), it fits the problem perfectly. Had you doubled the dataset and skipped feature engineering, LightGBM or XGBoost would likely have overtaken it.